

简书

首页

下载APP

会员

IT技术

搜索

Q

Aa

beta

登录

注册

Flutter高级面试题&答案



Qphine

总资产2

关注

【OpenHarmony应用开发】hdc常用命令大全

阅读 1,315

【OpenHarmony应用开发】定制化Launcher

阅读 469

热门故事

前世渣男把我迷晕后送到新科状元床上！

你看过最恐怖的悬疑小说是哪部？

妖艳女主播专挑凶宅直播跳舞结果.....

我捡回来的奶狗弟弟变身狼狗！遭不住

穿成王宝钏： 做大唐第一渣女

推荐阅读

落葵

阅读 2,049

年薪百万不一定实用！

阅读 2,007

太冲穴—人体出气筒

阅读 2,317

忙忙碌碌无功用

阅读 583

心无法心静了

阅读 1,079



Qphine

关注

1

2022.04.23 10:42:06 字数 4,279 阅读 10,444



1、Dart是值传递还是引用传递？



赏 t是值传递。

每次调用函数，传递过去的都是对象的内存地址，而不是这个对象的复制。

描述Flutter的核心渲染模块三棵树

WidgetTree:存放渲染内容、它只是一个配置数据结构，创建是非常轻量的，在页面刷新的过程中随时会重建

Element 是分离 WidgetTree 和真正的渲染对象的中间层，WidgetTree 用来描述对应的Element 属性,同时持有Widget和RenderObject，存放上下文信息，通过它来遍历视图树，支撑UI结构。

RenderObject (渲染树)用于应用界面的布局和绘制，负责真正的渲染，保存了元素的大小，布局等信息，实例化一个 RenderObject 是非常耗能的

当应用启动时 Flutter 会遍历并创建所有的 Widget 形成 Widget Tree，通过调用 Widget 上的createElement() 方法创建每个 Element 对象，形成 Element Tree。最后调用 Element 的createRenderObject() 方法创建每个渲染对象，形成一个 Render Tree。

3、flutter 中Widget的分类

1、组合类：StatelessWidget和StatefulWidget

2、代理类：InheritedWidget、ParentDataWidget

InheritedWidget一般用于状态共享，如Theme、Localizations、MediaQuery等，都是通过它实现共享状态，这样我们可以通过 context 去获取共享的状态，比如 ThemeData theme = Theme.of(context);

3、绘制类：RenderObjectWidget

RenderObject 的布局相关方法调用顺序是：layout -> performResize -> performLayout -> markNeedsPaint

4、mixin extends implement 之间的关系？

继承（关键字 extends）、混入 mixins（关键字 with）、接口实现（关键字 implements）。

这三者可以同时存在，前后顺序是extends -> mixins -> implements。

写下你的评论...



评论3



赞19



在Flutter中，Mixins是一种在多个类层次结构中复用类代码的方法。mixins的对象是类，mixins绝不是继承，也不是接口，而是一种全新的特性，可以mixins多个类，mixins的使用需要满足一定条件。

使用mixins的条件：

mixins类只能继承自object

mixins类不能有构造函数

一个类可以mixins多个mixins类

可以mixins多个类，不破坏Flutter的单继承

5、简述Dart语言特性

在Dart中，一切都是对象，所有的对象都是继承自Object

Dart是强类型语言，但可以用var或 dynamic来声明一个变量，Dart会自动推断其数据类型,dynamic类似c#

没有赋初值的变量都会有默认值null

Dart支持顶层方法，如main方法，可以在方法内部创建方法

Dart支持顶层变量，也支持类变量或对象变量

Dart没有public protected private等关键字，如果某个变量以下划线（_）开头，代表这个变量在库中是私有的

6、Dart 中的级联操作符

Dart 当中的「..」意思是「级联操作符」，为了方便配置而使用。「..」和「.」不同的是调用「..」后返回的相当于是 this，而「.」返回的则是该方法返回的值。

7、Dart 的单线程模型是如何运行的？

Dart 在单线程中是以消息循环机制来运行的，包含两个任务队列，一个是“微任务队列” microtask queue，另一个叫做“事件队列” event queue。

当Flutter应用启动后，消息循环机制便启动了。

按照先进先出的顺序逐个执行 微任务队列 中的任务，当所有 微任务队列 执行完后便开始执行 事件队列 中的任务，事件任务执行完后再去执行微任务，如此循环往复；

8、await for 与 stream流

```
1 | Stream<String> stream = new Stream<String>.fromIterable(['1', '2', '3', '4']);
2 | main() async{
3 |   print('start');
4 |   await for(String s in stream){
5 |     print(s);
6 |   }
7 |   print('end..');
8 | }
```

输出

```
1 | start
2 | 1
3 | 2
4 | 3
5 | 4
6 | end..
```

await for一般用在直到Stream什么时候完成，并且必须等待传递完成之后才能使用，不然就会一直阻塞。

9、Stream 与 Future是什么关系？

- 1、Future 表示稍后获得的一个数据，所有异步的操作的返回值都用 Future 来表示。
- 2、Future 只能表示一次异步获得的数据。
- 3、Stream 表示多次异步获得的数据。比如界面上的按钮可能会被用户点击多次，按钮上的点击事件（onClick）就是一个 Stream。
- 4、Future将返回一个值，而Stream将返回多次值。
- 5、Dart 中统一使用 Stream 处理异步事件流。

10、Stream 有哪两种订阅模式？分别是怎么调用的？

Stream有两种订阅模式：单订阅(single) 和 多订阅（broadcast）。单订阅就是只能有一个订阅者，而广播是可以有多个订阅者。这就有点类似于消息服务（Message Service）的处理模式。单订阅类似于点对点，在订阅者出现之前会持有数据，在订阅者出现之后就才转交给它。而广播类似于发布订阅模式，可以同时有多个订阅者，当有数据时就会传递给所有的订阅者，而不管当前是否已有订阅者存在。

Stream 默认处于单订阅模式，所以同一个 stream 上的 listen 和其它大多数方法只能调用一次，调用第二次就会报错。但 Stream 可以通过 transform() 方法（返回另一个 Stream）进行连续调用。通过 Stream.asBroadcastStream() 可以将一个单订阅模式的 Stream 转换成一个多订阅模式的 Stream，isBroadcast 属性可以判断当前 Stream 所处的模式。

11、Flutter中的Widget、State、Context 的核心概念？是为了解决什么问题？

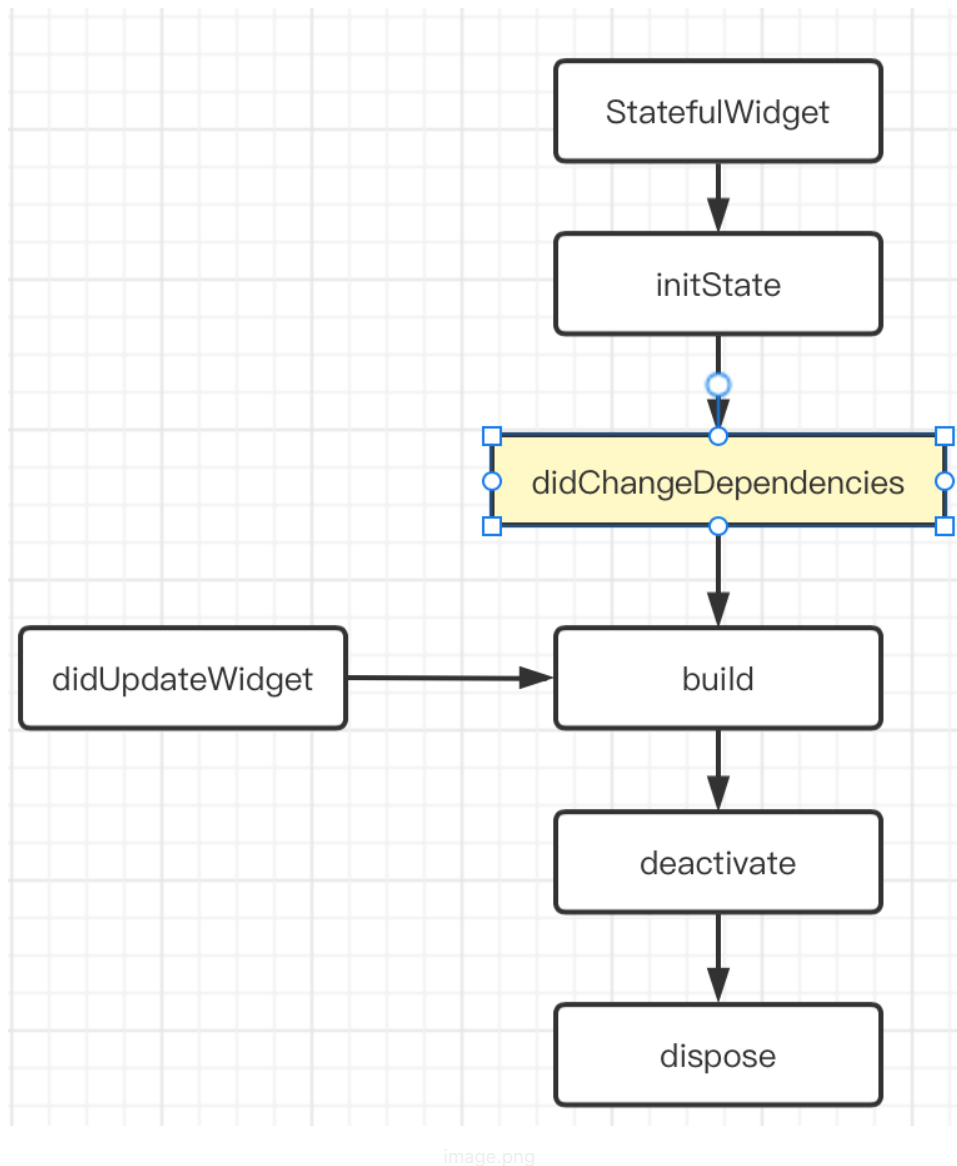
主要是为了解决多个部件之间的交互和部件自身状态的维护。

- 1、Widget: 在Flutter中，几乎所有东西都是Widget。将一个Widget想象为一个可视化的组件（或与应用可视化方面交互的组件），当你需要构建与布局直接或间接相关的任何内容时，你正在使用Widget。
- 2、Widget树: Widget以树结构进行组织。包含其他Widget的widget被称为父Widget(或widget容器)。包含在父widget中的widget被称为子Widget。
- 3、Context: 仅仅是已创建的所有Widget树结构中的某个Widget的位置引用。简而言之，将context作为widget树的一部分，其中context所对应的widget被添加到此树中。一个context只从属于一个widget，它和widget一样是链接在一起的，并且会形成一个context树。
- 4、State: 定义了StatefulWidget实例的行为，它包含了用于“交互/干预”Widget信息的行为和布局。应用于State的任何更改都会强制重建Widget。

12、Dart异步编程中的 Future关键字？

Dart中，执行一个异步任务使用Future来处理。在 Dart 的每一个 Isolate 当中，执行的优先级为：Main > MicroTask > EventQueue。

13、Flutter 中的生命周期



14、Widget 唯一标识Key

GlobalKey：确保生成的Key在整个应用中唯一，是很昂贵的，允许element在树周围移动或变更父节点而不会丢失状态；

LocalKey

UniqueKey

ObjectKey

15、Flutter是怎么完成组件渲染的？

在计算机系统中，图像的显示需要CPU、GPU和显示器一起配合完成CPU负责图像数据计算，GPU负责图像数据渲染，而显示器则负责最终图像显示。CPU把计算好的、需要显示的内容交给GPU，由GPU完成渲染后放入帧缓冲区，随后视频控制器根据垂直同步信号以每秒60次的速度，从帧缓冲区读取帧数据交由显示器完成图像显示。操作系统在呈现图像时遵循

了这种机制。

而Flutter作为跨平台开发框架也采用了这种底层方案，UI线程使用Dart语言来构建视图结构数据，这些数据会在GPU线程进行图层合成，随后交给图像渲染引擎Skia加工成GPU数据，而这些数据会通过OpenGL最终提供给GPU渲染。

可以看到Flutter用了计算机最基本的图像渲染技术，摒弃其他一些通道和过程，用最直接的方式完成了图形显示，自然性能也就得到了保障。

16、PlatformView 以及其原理

Flutter 中通过 PlatformView 可以嵌套原生 View 到 Flutter UI 中，这里面其实是使用了 Presentation + VirtualDisplay + Surface 等实现的，大致原理：

使用了类似副屏显示的技术，VirtualDisplay 类代表一个虚拟显示器，调用 DisplayManager 的 createVirtualDisplay() 方法，将虚拟显示器的内容渲染在一个 Surface 控件上，然后将 Surface 的 id 通知给 Dart，让 engine 绘制时，在内存中找到对应的 Surface 画面内存数据，然后绘制出来，实时控件截图渲染显示技术。

17、Flutter 线程管理模型

Flutter Engine层会创建一个Isolate，并且Dart代码默认就运行在这个主Isolate上。必要时可以使用spawnUri和spawn两种方式来创建新的Isolate，在Flutter中，新创建的Isolate由Flutter进行统一的管理。

事实上，Flutter Engine自己不创建和管理线程，Flutter Engine线程的创建和管理是Embedder负责的，Embedder指的是将引擎移植到平台的中间层代码。

Flutter 中存在的四大线程：分别为 UI Runner、GPU Runner、IO Runner， Platform Runner（原生主线程），

在 Flutter 中可以通过 isolate 或者 compute 执行真正的跨线程异步操作。

18、Flutter状态管理

Flutter的状态可以分为全局状态和局部状态两种。常用的状态管理有ScopedModel、BLoC、Redux / FishRedux和Provider。

状态管理基本都是基于InheritedWidget封装的用于Widget树的数据传递与共享的一套框架

Provider是继承于InheritProvider，而InheritProvider本质上是一个InheritWidget，所以Provider本质上是依托于InheritProvider的机制来实现的widget树的状态共享。

[Flutter中Provider和Redux状态管理简述](#)

19、isolate是怎么进行通信和实例化的？

- 1、isolate实际就是一个隔离的Dart执行的上下文环境(或者容器)
- 2、isolate是有自己的内存和单线程控制的事件循环
- 3、isolate之间的内存存在逻辑上是隔离的，不像Java一样是共享内存的
- 4、任何Dart程序的并发都是运行多个isolate的结果。Dart没有共享内存的并发；

isolate线程之间的通信主要通过port来进行，这个port消息传递过程是异步的。

```

1 //实现newIsolate与rootIsolate互相通信
2 import 'dart:io';
3 import 'dart:isolate';
4 //定义一个新Isolate
5 late Isolate newIsolate;
6 //定义一个新IsolateSendPort, 该newIsolateSendPort需要让rootIsolate持有,
7 //这样在rootIsolate中就能利用newIsolateSendPort向newIsolate发送消息
8 late SendPort newIsolateSendPort;
9
10 void main() {
11   establishConn(); //建立连接
12 }
13
14 //特别需要注意:establishConn执行环境是rootIsolate
15 void establishConn() async {
16   //第1步: 默认执行环境下是rootIsolate, 所以创建的是一个rootIsolateReceivePort
17   ReceivePort rootIsolateReceivePort = ReceivePort();
18   //第2步: 获取rootIsolateSendPort
19   SendPort rootIsolateSendPort = rootIsolateReceivePort.sendPort;
20   //第3步: 创建一个newIsolate实例, 并把rootIsolateSendPort作为参数传入到newIsolate中, 为的是让
21   newIsolate = await Isolate.spawn(createNewIsolateContext, rootIsolateSendPort); //注意
22   //第7步: 通过rootIsolateReceivePort接收到来自newIsolate的消息, 所以可以注意到这里是await 因为!
23   //只不过这个接收到的消息是newIsolateSendPort, 最后赋值给全局newIsolateSendPort, 这样rootIsolate
24   newIsolateSendPort = await rootIsolateReceivePort.first;
25   //第8步: 建立连接后, 在rootIsolate环境下就能向newIsolate发送消息了
26   sendMessageToNewIsolate(newIsolateSendPort);
27 }
28
29 //特别需要注意:sendMessageToNewIsolate执行环境是rootIsolate
30 void sendMessageToNewIsolate(SendPort newIsolateSendPort) async {
31   ReceivePort rootIsolateReceivePort = ReceivePort(); //创建专门应答消息rootIsolateReceive
32   SendPort rootIsolateSendPort = rootIsolateReceivePort.sendPort;
33   newIsolateSendPort.send(['this is from root isolate: hello new isolate!'], rootIsolate
34   //第11步: 监听来自newIsolate的消息
35   print(await rootIsolateReceivePort.first);
36 }
37
38 //特别需要注意:createNewIsolateContext执行环境是newIsolate
39 void createNewIsolateContext(SendPort rootIsolateSendPort) async {
40   //第4步: 注意callback这个函数执行环境就会变为newIsolate, 所以创建的是一个newIsolateReceivePort
41   ReceivePort newIsolateReceivePort = ReceivePort();
42   //第5步: 获取newIsolateSendPort, 有人可能疑问这里为啥不是直接让全局newIsolateSendPort赋值, 注!
43   SendPort newIsolateSendPort = newIsolateReceivePort.sendPort;
44   //第6步: 特别需要注意这里, 这里是利用rootIsolateSendPort向rootIsolate发送消息, 只不过发送消息是
45   rootIsolateSendPort.send(newIsolateSendPort);
46   //第9步: newIsolateReceivePort监听接收来自rootIsolate的消息
47   receiveMsgFromRootIsolate(newIsolateReceivePort);
48 }
49
50 //特别需要注意:receiveMsgFromRootIsolate执行环境是newIsolate
51 void receiveMsgFromRootIsolate(ReceivePort newIsolateReceivePort) async {
52   var messageList = (await newIsolateReceivePort.first) as List;
53   print('${messageList[0] as String}');
54   final messageSendPort = messageList[1] as SendPort;
55   //第10步: 收到消息后, 立即向rootIsolate 发送一个回复消息
56   messageSendPort.send('this is reply from new isolate: hello root isolate!');
57 }

```

代码摘自<https://zhuanlan.zhihu.com/p/355315785>

20、Future还是isolate场景分析？

- 1、如果一段代码不会被中断，那么就直接使用正常的同步执行就行。
- 2、如果代码段可以独立运行而不会影响应用程序的流畅性，建议使用 Future （需要花费几毫秒时间）
- 3、如果繁重的处理可能要花一些时间才能完成，而且会影响应用程序的流畅性，建议使用 isolate （需要几百毫秒）

下面列出一些使用 isolate 的具体场景:

- 1、JSON解析: 解码JSON, 这是HttpRequest的结果, 可能需要一些时间, 可以使用封装好的 isolate 的 compute 顶层方法。
- 2、加解密: 加解密过程比较耗时
- 3、图片处理: 比如裁剪图片比较耗时
- 4、从网络中加载大图

21、Flutter 是如何与原生Android、iOS进行通信的?

Flutter 通过 PlatformChannel 与原生进行交互, 其中 PlatformChannel 分为三种:

BasicMessageChannel : 用于传递字符串和半结构化的信息。

MethodChannel : 用于传递方法调用 (method invocation) 。

EventChannel : 用于数据流 (event streams) 的通信。

22、Flutter 绘制流程

Flutter只关心向 GPU提供视图数据, GPU的 VSync信号同步到 UI线程, UI线程使用 Dart来构建抽象的视图结构, 这份数据结构在 GPU线程进行图层合成, 视图数据提供给 Skia引擎渲染为 GPU数据, 这些数据通过 OpenGL或者 Vulkan提供给 GPU。

23、Flutter 的热重载

Flutter 的热重载是基于 JIT 编译模式的代码增量同步。由于 JIT 属于动态编译, 能够将 Dart 代码编译成生成中间代码, 让 Dart VM 在运行时解释执行, 因此可以通过动态更新中间代码实现增量同步。

热重载的流程可以分为 5 步, 包括: 扫描工程改动、增量编译、推送更新、代码合并、Widget 重建。Flutter 在接收到代码变更后, 并不会让 App 重新启动执行, 而只会触发 Widget 树的重新绘制, 因此可以保持改动前的状态, 大大缩短了从代码修改到看到修改产生的变化之间所需要的时间。

另一方面, 由于涉及到状态的保存与恢复, 涉及状态兼容与状态初始化的场景, 热重载是无法支持的, 如改动前后 Widget 状态无法兼容、全局变量与静态属性的更改、main 方法里的更改、initState 方法里的更改、枚举和泛型的更改等。

可以发现, 热重载提高了调试 UI 的效率, 非常适合写界面样式这样需要反复查看修改效果的场景。但由于其状态保存的机制所限, 热重载本身也有一些无法支持的边界。

24、Flutter 热更新

Android:

利用原生框架更新, 实际上就是更新Flutter框架相关的二进制。Flutter应用发布出来的产物主要包括 libflutter.so, libapp.so, flutterAssets, 这样, 就可以通过Android端原生平台网络请求, 动态下发并加载这些产物, 从而实现热更新。

iOS: 苹果商店不允许动态下发和加载二进制产物, 包括动态库之类的;

25、Flutter 动态化方案

基本思路:

通过定义统一的描述语言(JSON来表示结构、样式和行为), 然后通过可视化平台来拖拽出 JSON模板, 最后将JSON模板下发到Flutter App, Flutter App内置了JS模板引擎以及DSL解析引擎, 由它们将DSL解析映射为Flutter Widgets或者渲染对象。

Flutter动态化整体架构

总体架构上分为四大部分。

第一部分是可视化搭建平台，负责开发DSL页面和配置数据。

第二部分是低代码服务中台，提供组件保存、页面发布和数据加工能力。

第三部分是面向端的接口服务，包括模板和数据接口。

第四部分是端，这块是核心重点，端上需要支持一整套DSL的解析和渲染映射，并且要做好相应的优化，以保证渲染性能和效率。

- 1、[美团外卖Flutter动态化实践](#)
- 2、[携程App 首页动态化探索](#)
- 3、[Flutter 动态化在最右 App 中的实践](#)
- 4、[Flutter 动态化热更新的思考与实践](#)
- 5、[NOW直播Flutter动态搜索列表页实现](#)
- 6、[Flutter动态化的方案对比及最佳实现-闲鱼](#)
- 7、[基于JavaScript 的MXFlutter](#)
- 8、[Flutter App动态化与可视化搭建方案设计](#)



19人点赞 >



Android技术



更多精彩内容，就在简书APP



"小礼物走一走，来简书关注我"

赞赏支持

还没有人赞赏，支持一下



Qphine

总资产2 共写了8670字 获得24个赞 共9个粉丝

关注

写下你的评论...

全部评论 3

只看作者

按时间倒序 按时间正序



guiqiang107
3楼 08.22 09:57

我发现dart是值传递这个问题，都是百度千篇一律出来的结果。连值传递和引用传递都没区分清楚就贴出来了。有没有一种可能，基础数据类型是值传递，List,Map, Set, Class等是引用传递？



Codepgq
12.02 19:08

看到第一题就知道是别处抄来的了



任振铭
2楼 08.07 12:55

“dart是值传递。
每次调用函数，传递过去的都是对象的内存地址，而不是这个对象的复制。”

既然传递的是引用地址，为什么还是值传递，是不是矛盾了



被以下专题收入，发现更多相似内容



Flutter



flutter...



Flutter面试题



Flutter



Flutter



Android技术

推荐阅读

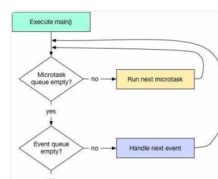
更多精彩内容 >

Flutter常见面试题

Flutter是Google推出的一套开源跨平台UI框架，可以快速地在Android、iOS和Web平台上构建高质...



GoldMask 阅读 6,044 评论 3 赞 36

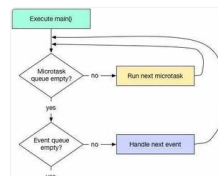


flutter面试题

1. Dart 当中的「..」表示什么意思？ Dart 当中的「..」意思是「级联操作符」，为了方便配置而使...



马修斯 阅读 12,882 评论 0 赞 21



Flutter面试题

1、Dart中var 与 dynamic的区别 2、const和final的区别 3、Dart中 ?? 与 ??=...



永不放弃_8eef 阅读 461 评论 0 赞 2

Flutter 1 Flutter是什么？Flutter是谷歌的移动UI框架，可以快速在iOS和Andro...

江河_ios 阅读 714 评论 0 赞 2

人就是孤独的，压力太大，缓解它带来的痛苦的唯一方式就是平静的接受它；摆脱痛苦的一种方式也只有努力，让自己蜕变...

码农ing 阅读 821 评论 2 赞 10

1、main()和runApp()函数在flutter的作用分别是什么？有什么关系吗？ main函数是类似于jav...

 雷霸龙 阅读 1,014 评论 1 赞 7



我印象里，身边的老人们，总喜欢把一句话挂在嘴边：“生活不如意事，十之八九！人嘛，总得，自己学会适应生活！”...

cf96731e55b0 阅读 1,404 评论 0 赞 4

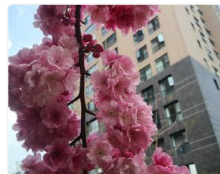
今天是第三周周一，三八女神节祝自己每天开心一点，工作顺心顺意 起床：4：25 就寝：23：56 天气：雨天 心情：...

 QXCLZ 阅读 931 评论 1 赞 3



张利平2021.3.6「学习《情绪按钮》第20天收获： [太阳]今天学习内容： 第七章《情绪的来源》（五）情绪的来...

张利平专注国学教育139876 阅读 4,299 评论 2 赞 3



这周的作文题目是：假设你现在的火车上，对面坐着一个漂亮的异性，去构思一段故事，想想接下来会发生什么。那么接下来我的...

2077516 阅读 2,313 评论 3 赞 0

人们问爱因斯坦为何能在1905年提出那么多改变人类认识世界的理论，他谦虚地回答道：“并不是我很聪明，只是我和问题...”

 世界和平_众生安康 阅读 4,824 评论 0 赞 10



我一点也不开心。负面情绪已经完全超越了理智。我知道自己要去想办法解决问题，可是我只想逃离。难受极了！

Woody是多么的寂寞 阅读 552 评论 1 赞 0

